USE EFFECT

THE USE EFFECT HOOK

Made with GAMMA

Agenda: What We'll Cover Today

- Why system design interviews matter
- The overall system design process / framework
- Deep dive: Clarify & Define Requirements (Functional & Non-Functional)
- Estimation & Capacity Planning (brief overview)
- High-Level Design Overview
- Case Study: Designing a popular system
- Key Takeaways

Made with GAMMA

What are React Hooks|?

- Hooks allow functions to have access to state and other React features without using classes.
- State refers to application data or properties to be changed.

- They provide a more direct API to React concepts like props, state, context, refs, and lifecycle.
- You must import Hooks from react
- Examples of React Hooks, UseState,UseEffect, useContext,useRef,useReducer,React Custom Hooks, etc

-

Hook Rules

Three 3 rules for hooks:

- Hooks can only be called inside React function components.
- Hooks can only be called at the top level of a component.
- Hooks cannot be conditional

Made with GAMMA

main.jsx
index.html

```
import { useState } from 'react';
import { createRoot } from 'react-dom/client';

function FavoriteColor() {
 const [color, setColor] = useState("red");
 return (
   <>
     <h1>My favorite color is {color}!</h1>
<button
       type="button"
       onClick={() => setColor("blue")}
     >Blue</button>
     <button
       type="button"
       onClick={() => setColor("red")}
     >Red</button>
     <button
       type="button"
       onClick={() => setColor("pink")}
     >Pink</button>
     <button
       type="button"
       onClick={() => setColor("green")}
    >Green</button>
   </>
  );
}
createRoot(document.getElementById('root')).render(
```

RUN

`useState` accepts an initial state and returns two values:

- **The current state.**
- **A function that updates the state**

The first value, `color`, is our current state.

The second value, `setColor`, is the function that is used to update our state.

These names are variables that can be named anything you would like.

Lastly, we set the initial state to "red": `useState("red")`

Made with GAMMA

3. The History: What came before?

The old way (before 2019)

Before Hooks were introduced in **React 16.8 (2019)**,

we lived in the era of **Class Components**.

If you wanted a component to have state,

you couldn't use a simple function.

You had to write a full ES6 Class that extended `React.Component`.

Feature	Class Components (The Past)	Hooks (The Present)
State	Handled via this.state and this.setState().	Handled via useState().
Lifecycle	Methods like componentDidMount and componentWillUnmount.	Handled via the useEffect() hook.
Logic Sharing	Required complex patterns like Higher-Order Components (HOCs).	Simply create a Custom Hook.
Complexity	High. You had to understand how the this keyword works in JS.	Low. It's just standard functional programming.

Made with GAMMA

WHAT IS USE EFFECT?

The useEffect Hook allows you to perform side effects in your components.

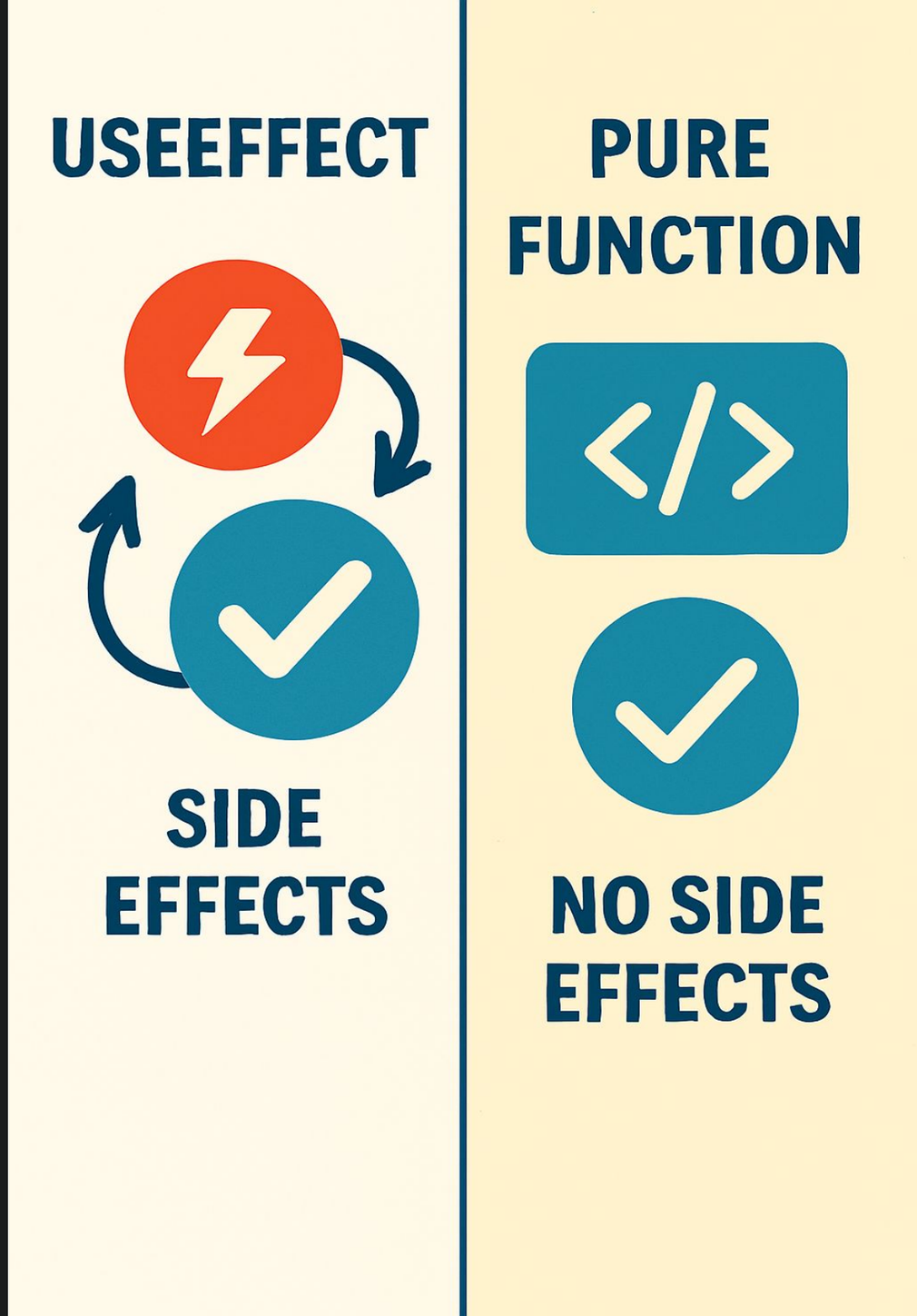
Some examples of side effects are: fetching data, directly updating the DOM, and timers.

useEffect accepts two arguments. The second argument is optional.

useEffect(<function>, <dependency>)

hThere are several ways to control when side effects run.

We should always include the second parameter which accepts an array. We can optionally pass dependencies to useEffect in this array.



Effect Cleanup

Some effects require cleanup to reduce memory leaks.

Timeouts, subscriptions, event listeners, and other effects that are no longer needed should be disposed.

We do this by including a return function at the end of the `useEffect` Hook.

Made with GAMMA

Example Cleanup:

Clean up the timer at the end of the `useEffect` Hook:

```
import { useState, useEffect } from 'react';
import { createRoot } from 'react-dom/client';

function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    let timer = setTimeout(() => {
      setCount((count) => count + 1);
    }, 1000);

    return () => clearTimeout(timer)
  }, []);

  return <h1>I've rendered {count} times!</h1>;
}

createRoot(document.getElementById('root')).render(
  <Timer />
```

Made with GAMMA

Pure Function vs. useEffect (Side Effects)

#	Feature	Pure Function (The Render)	useEffect (The Side Effect)
1	**Predictability**	**Deterministic:** Same props always result in the exact same JSX output.	**Non-Deterministic:** Depends on external factors (API response, Timer, DOM state).
2	**Timing**	Runs **during** the render phase (while React is calculating the UI).	Runs **after** the commit phase (after the browser has painted the screen).
3	**Dependencies**	Uses any variable available in scope immediately.	Only re-runs if specific values in the **dependency array** change.
4	**Outside World**	**Isolated:** Cannot touch the DOM, Cookies, or Databases directly.	**Interactive:** Designed specifically to sync with APIs, Subscriptions, and the DOM.
5	**Cleanup**	No concept of cleanup; the function finishes and is garbage collected.	Provides a **Cleanup Function** to prevent memory leaks (e.g., clearInterval).
6	**Idempotency**	Must be **Idempotent**: Calling it 10 times should not change the system state.	**State-Changing:** Often used specifically to update state or trigger external events.
7	**Mathematical**	Represents a **Mapping** ($f(x) = \text{UI}$).	Represents a **Synchronization** (UI $\leftrightarrow$ External

Made with GAMMA

Summary: Key Takeaways

Requirements are King

Spend ample time clarifying functional and non-functional requirements. This is where top candidates shine.

Structured Approach

Follow a clear framework: Requirements → Estimation → High-Level Design → Deep Dive.

Trade-offs are Inevitable

Understand and articulate the compromises you make (e.g., consistency vs. availability).

Communicate Clearly

Articulate your thought process, assumptions, and design choices effectively.

Practice Makes Perfect

Work through common system design problems, focusing on the initial requirements phase.

Remember: Your ability to clarify requirements and drive the conversation sets you apart.

Made with GAMMA